

Unknown Title

Authenticating Git

Authenticating Git & GitHub

Authenticating GitHub is important to protect your account and your code from unauthorized access:

Username and Password with Two-Factor Authentication, or a Passkey

As the default authentication method, this requires you to enter your login credentials each time you want to access GitHub. Currently, this method only supports GitHub login, but not project cloning and pushing.

Username and Password with 2FA

- Requires you to enter your username, password, and a one-time code from your phone each time you want to authenticate.
- This adds an extra layer of security to your account, as even if someone has your username and password, they will not be able to log in without the one-time code.

Passkey

- A passkey is a unique digital credential that can be used to authenticate to websites and apps without the need for a password.
- To use a passkey, you will need to create one on your device and then register it with GitHub.
- Once you have registered a passkey with GitHub, you can use it to authenticate to GitHub without having to enter a username or password.

Personal Access Token

A personal access token (PAT) is a unique string that can be used to authenticate to GitHub on your behalf. PATs can be created with different permissions and scopes, so you can control what actions they can perform. They can also be used to grant access to your repositories to third-party apps and services.

Steps to Create a GitHub Personal Access Token (PAT)

1. Go to your GitHub account settings and click the **Personal Access Tokens** tab.
2. Click the **Generate new token** button.
3. Enter a **name** and **description** for your PAT.
4. Select the **permissions** and **scopes** that you want your PAT to have.

5. Click the **Generate token** button.

Your PAT will be displayed in a text box. Copy and store your PAT in a secure location. You will need this PAT to authenticate to GitHub using the command line or third-party apps and services.

How to Clone a Repository Using a Personal Access Token (PAT)

1. Open a terminal window.
2. Navigate to the directory where you want to clone the repository.
3. Clone the repository using the following command:

```
git clone https://github.com/<username>/<repository>.git --token <pat>
```

Replace `<username>` with your GitHub username, `<repository>` with the name of the repository you want to clone, and `<pat>` with your PAT.

For example, to clone the `my-repo-001` repository owned by the user `betascribbles`, you would use the following command:

```
git clone https://github.com/betascribbles/my-repo-001.git --token <pat>
```

1. Once the repository has been cloned, you can navigate to it using the `cd` command and start working on the code.

How to Push Changes to a Repository Using a PAT

1. Open a terminal window.
2. Navigate to the directory where you want to push the changes from.
3. Stage the changes that you want to push using the `git add` command.
4. Commit the changes using the `git commit` command.
5. Push the changes to the remote repository using the following command:

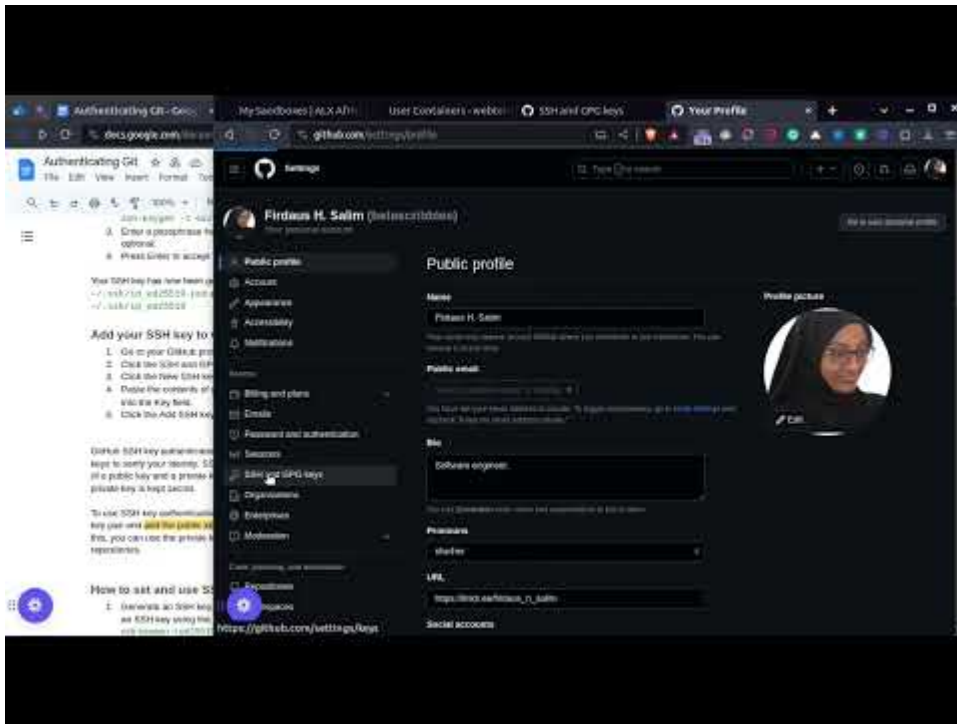
```
git push https://github.com/<username>/<repository>.git --token <pat>
```

Replace `<username>` with your GitHub username, `<repository>` with the name of the repository you want to push to, and `<pat>` with your PAT.

For example, to push the changes to the `my-repo-001` repository owned by the user `betascribbles`, you would use the following command:

```
git push https://github.com/betascribbles/my-repo-001.git --token <pat>
```

SSH Key



<https://youtu.be/X8Mp-s6ZQVo>

Generate an SSH Key

1. Open a terminal window.
2. Generate a new SSH key with the following command:

```
ssh-keygen -t ed25519
```

1. Enter a passphrase for your key when prompted. The passphrase is optional.
2. Press Enter to accept the default file location for your key.

Your SSH key has now been generated. The public key is stored in the file `~/.ssh/id_ed25519.pub` and the private key is stored in the file `~/.ssh/id_ed25519`.

Add Your SSH Key to GitHub

1. Go to your GitHub profile settings.
2. Click the **SSH and GPG keys** tab.
3. Click the **New SSH key** button.
4. Paste the contents of your public key file (`~/.ssh/id_ed25519.pub`) into the Key field.
5. Click the **Add SSH key** button.

GitHub SSH key authentication is a method of authentication that uses SSH keys to verify your identity. SSH keys are a cryptographic key pair that consists of a public key and a private key. The public key is shared with others, while the private key is kept secret.

To use SSH key authentication with GitHub, you will need to generate an SSH key pair and add the public key to your GitHub account. Once you have done this, you can use the private key to authenticate to GitHub and access your repositories.

How to Set and Use SSH Keys

1. Generate an SSH key. If you don't already have one, you can generate an SSH key using the following command:

```
ssh-keygen -t ed25519 -C "your_email@example.com"
```

1. Add your SSH key to GitHub. Go to your **GitHub account settings** and click the **SSH and GPG keys** tab. Click the **New SSH key** button and paste the contents of your public key file (`~/.ssh/id_ed25519.pub`) into the Key field. Click the **Add SSH key** button.
2. Clone a GitHub repository using SSH. To clone a GitHub repository using SSH, use the following command:

```
git clone git@github.com:username/repository.git
```

Replace `username` with your GitHub username and `repository` with the name of the repository you want to clone.

1. Push changes to a GitHub repository using SSH. To push changes to a GitHub repository using SSH, use the following command:

```
git push git@github.com:username/repository.git
```

Replace `username` with your GitHub username and `repository` with the name of the repository you want to push to.

Resources

- [GitHub auth documentation](#)
- [Support for password authentication was removed GitHub Fixed using Token \(August 13, 2021\) - Linux by Code With Arjun](#)
- [How to generate GitHub PAT](#)

Copyright © 2025 ALX, All rights reserved.